AI ASSISTED CODING

LAB ASSIGNMENT-1.1

NAME: M. Shashi

HTNO: 2303A51028

Lab 1: Environment Setup – GitHub Copilot and VS Code Integration + Understanding AI-assisted Coding Workflow

Task 0

● Install and configure GitHub Copilot in VS Code. Take screenshots of each step.

Task 1: AI-Generated Logic Without Modularization (Factorial without Functions)

• Scenario

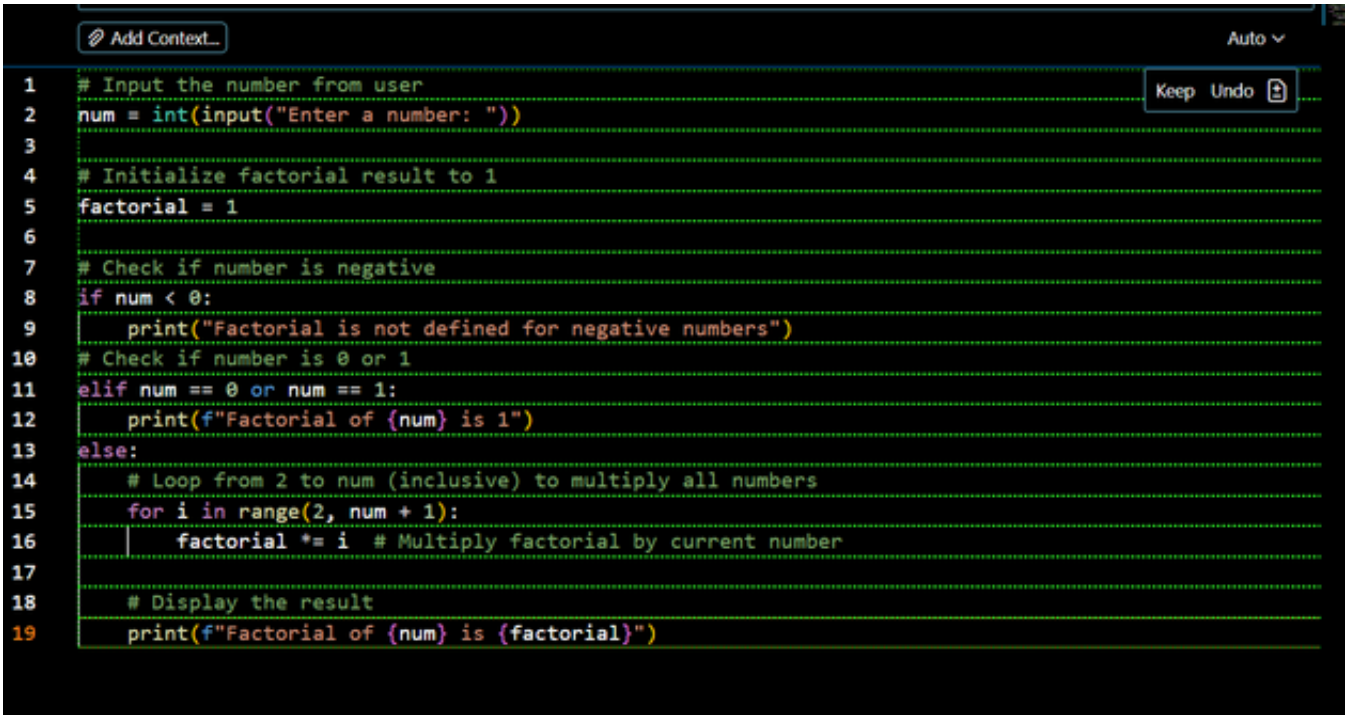
You are building a small command-line utility for a startup intern onboarding task. The program is simple and must be written quickly without modular design.

• Task Description

Use GitHub Copilot to generate a Python program that computes a

mathematical product-based value (factorial-like logic) directly in the main execution flow, without using any user-defined functions.

➢ Copilot's suggestions



➢ Sample input/output screenshots

```python
num = int(input("Enter a number: "))

# Initialize factorial result to 1
factorial = 1

# Check if number is negative
if num < 0:
    print("Factorial is not defined for negative numbers")
# Check if number is 0 or 1
elif num == 0 or num == 1:
    print(f"Factorial of {num} is 1")
else:
    # Loop from 2 to num (inclusive) to multiply all numbers
    for i in range(2, num + 1):
        factorial *= i  # Multiply factorial by current number

    # Display the result
    print(f"Factorial of {num} is {factorial}")
```

OBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA> & C:/Users/varsh/AppData/Lo
n.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignment-1.py
Enter a number: 10
Factorial of 10 is 3628800
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

```
Enter a number: -5
Factorial is not defined for negative numbers
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
on.exe c:/Users/varsh/OneDrive/ドキュメント/third
Enter a number: 0
Factorial of 0 is 1
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
on.exe c:/Users/varsh/OneDrive/ドキュメント/third
Enter a number: 1
Factorial of 1 is 1
```

➢ **Brief reflection (5–6 lines):**

GitHub Copilot was very helpful for a beginner as it quickly generated correct and readable code with minimal effort. It understood the requirement of avoiding functions and used only loops and variables as instructed. The generated logic was simple, clean, and easy to understand. Copilot also followed basic best practices like input validation for negative numbers. This tool can significantly speed up learning and development for new programmers. However, understanding the logic manually is still important to avoid over-dependency.

➢ **How helpful was Copilot for a beginner?**

GitHub Copilot is very helpful for beginners because it quickly generates working code from simple prompts. It reduces the time spent thinking about syntax and structure. Beginners can focus more on understanding logic rather than writing code from scratch.

➢ **Did it follow best practices automatically?**

Yes, Copilot mostly follows basic best practices automatically.It uses clean variable names, proper loops, and simple input handling.However, beginners should still review the code to fully understand and improve it.

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖ Scenario

Your team lead asks you to review AI-generated code before committing it to a shared repository.

❖ Task Description

Analyze the code generated in Task 1 and use Copilot again to:

➢ Reduce unnecessary variables

➢ Improve loop clarity

➢ Enhance readability and efficiency

❖ Expected Deliverables

➢ **Original AI-generated code**

```python
# Input the number from user
num = int(input("Enter a number: "))
# Initialize factorial result to 1
factorial = 1
# Check if number is negative
if num < 0:
    print("Factorial is not defined for negative numbers")
# Check if number is 0 or 1
elif num == 0 or num == 1:
    print(f"Factorial of {num} is 1")
else:
    # Loop from 2 to num (inclusive) to multiply all numbers
    for i in range(2, num + 1):
        factorial *= i  # Multiply factorial by current number

    # Display the result
    print(f"Factorial of {num} is {factorial}")
```

➢ **Optimized version of the same code**

```python
num = int(input("Enter a number: "))
if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    result = 1  # Stores the factorial value
    for i in range(1, num + 1):  # Loop directly from 1 to num
        result *= i
    print(f"Factorial of {num} is {result}")
```

```
Enter a number: -5
Factorial is not defined for negative numbers
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
on.exe c:/Users/varsh/OneDrive/ドキュメント/third
Enter a number: 0
Factorial of 0 is 1
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIA
on.exe c:/Users/varsh/OneDrive/ドキュメント/third
Enter a number: 1
Factorial of 1 is 1
```

➢ Side-by-side comparison

1. Reduced Unnecessary Conditions

Previous code had a separate check for 0 and 1.

Optimized code removes it because looping from 1 naturally handles both cases. This reduces branching and improves clarity.

2. Improved Loop Clarity

Previous code looped from 2 and relied on pre-initialization.

Optimized code loops cleanly from 1 to num, making logic easier to understand for beginners.

3. Better Readability

Straightforward control flow. Easier to read, debug, and modify.

➢ Written explanation:

▪ What was improved?

The optimized version removes unnecessary conditional checks and simplifies the overall control flow. It eliminates the special-case handling for 0 and 1 by using a single loop that naturally covers all valid cases. The loop structure was made clearer by iterating directly from 1 to num. Variable usage was minimized to keep only what is essential for computation.

▪ Why the new version is better (readability, performance, maintainability.

Readability: The code is shorter and easier to understand, especially for beginners, because it has fewer branches and a straightforward loop.**Performance:** Although factorial computation is already efficient, removing extra conditions slightly reduces execution steps.**Maintainability:** With simpler logic and fewer cases to handle, the code is easier to modify, debug, and extend in the future.

Task 3: Modular Design Using AI Assistance (Factorial with Functions)

❖ Scenario

The same logic now needs to be reused in multiple scripts.

❖ Task Description

Use GitHub Copilot to generate a modular version of the program by:

➢ Creating a user-defined function

➢ Calling the function from the main block

```python
# Generate the above code by user-defined function must call the function from the main block using meaningful function and variable names, include inline comments
def calculate_factorial(number):
    """Calculate the factorial of a given number."""
    if number < 0:
        return "Factorial is not defined for negative numbers"
    result = 1  # Initialize result to store factorial value
    for i in range(1, number + 1):  # Loop from 1 to number
        result *= i  # Multiply result by current number
    return f"Factorial of {number} is {result}"
if __name__ == "__main__":
    user_input = int(input("Enter a number: "))  # Get user input
    print(calculate_factorial(user_input))  # Call the function and print the result
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
_assignment-1.py
Enter a number: 5
Factorial of 5 is 120
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignm
Enter a number: -5
Factorial is not defined for negative numbers
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignm
Enter a number: 0
Factorial of 0 is 1
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/lab_assignm
Enter a number: 1
Factorial of 1 is 1
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Task 4: Comparative Analysis – Procedural vs Modular AI Code (With vs Without Functions)

❖ Scenario

As part of a code review meeting, you are asked to justify design choices.

❖ Task Description

Compare the non-function and function-based Copilot-generated

programs on the following criteria:

➢ Logic clarity

➢ Reusability

➢ Debugging ease

➢ Suitability for large projects

➢ AI dependency risk

Criteria	Non-Function Program	Function-Based Program
Logic Clarity	Logic is written directly in the main flow, which is easy to understand for very small programs but confusing when the program becomes bigger.	Logic is encapsulated inside a function, making the program flow cleaner and easier to follow.
Reusability	Code cannot be reused without copying and pasting the entire logic.	The function can be reused multiple times by calling it with different inputs.
Debugging Ease	Debugging is harder because all logic is in one place.	Easier to find and fix errors because the code is separated into a function.
Suitability for Large Projects	Not suitable for large projects due to poor structure and scalability.	Good for large projects because the code can be easily expanded and maintained
AI Dependency Risk	Higher risk of blindly accepting AI-generated logic without understanding structure.	Lower risk because functions encourage logical thinking and better understanding of code flow.

The non-function version is acceptable for quick scripts or beginner practice.The function-based version is clearly superior for readability, reuse, debugging, and real-world software development.Copilot performs well in both cases, but modular code reduces long-term dependency on AI-generated solutions.

Task 5: AI-Generated Iterative vs Recursive Thinking

❖ Scenario

Your mentor wants to test how well AI understands different computational paradigms.

❖ Task Description

Prompt Copilot to generate:

An iterative version of the logic

A recursive version of the same logic

❖ Expected Deliverables

Two AI-generated implementations

Execution flow explanation (in your own words)

Comparison covering:

➢ Readability

➢ Stack usage

➢ Performance implications

➢ When recursion is not recommended.

```python
# Generate the above code with iterative and recursive methods to calculate factorial, include inline comments
def calculate_factorial_iterative(number):
    """Calculate the factorial of a given number using an iterative approach."""
    if number < 0:
        return "Factorial is not defined for negative numbers"
    result = 1  # Initialize result to store factorial value
    for i in range(1, number + 1):  # Loop from 1 to number
        result *= i  # Multiply result by current number
    return f"Factorial of {number} is {result} - Iterative Method"
def calculate_factorial_recursive(number):
    """Calculate the factorial of a given number using a recursive approach."""
    if number < 0:
        return "Factorial is not defined for negative numbers"
    elif number == 0 or number == 1:
        return 1  # Base case: factorial of 0 or 1 is 1
    else:
        return number * calculate_factorial_recursive(number - 1)  # Recursive case
if __name__ == "__main__":
    user_input = int(input("Enter a number: "))  # Get user input
    # Call the iterative function and print the result
    print(calculate_factorial_iterative(user_input))
    # Call the recursive function and print the result
    recursive_result = calculate_factorial_recursive(user_input)
    if isinstance(recursive_result, str):
        print(recursive_result)  # Print error message if any
    else:
        print(f"Factorial of {user_input} is {recursive_result}- Recursive Method")
```

Iterative Function:

The program starts execution from the main. It takes a number as input from the user. First, the iterative function is called:

It checks if the number is negative.

If valid, it uses a loop to multiply numbers from 1 to n.

The result is returned and printed.

Recursive Function:

It checks for a base case (0 or 1) and returns 1.

Otherwise, it calls itself with number - 1.

Each call waits until the base case is reached, then results are multiplied back.

Finally, both results are displayed.

OUTPUT:

```
Enter a number: 2
Factorial of 2 is 2 - Iterative Method
Factorial of 2 is 2- Recursive Method
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/
Enter a number: 1
Factorial of 1 is 1 - Iterative Method
Factorial of 1 is 1- Recursive Method
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/
Enter a number: -10
Factorial is not defined for negative numbers
Factorial is not defined for negative numbers
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/
Enter a number: 0
Factorial of 0 is 1 - Iterative Method
Factorial of 0 is 1- Recursive Method
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>
```

Readability

- The iterative method is easy to read and understand because it uses a simple loop. The recursive method is harder to understand since the function calls itself.

Stack Usage

- The iterative method does not use extra stack memory.
- The recursive method uses stack memory for each function call.

Performance Implications

- The iterative method is faster and more efficient, especially for large numbers. The recursive method is slower due to repeated function calls.

When Recursion is Not Recommended

- When the input number is very large.
- When memory usage needs to be low.
- When there is a risk of stack overflow.
- When a simple loop can solve the problem easily.
- Iteration is better for performance and safety, while recursion is mainly useful for understanding concepts.